MANUAL TÉCNICO DE INSTALACIÓN, CONFIGURACIÓN Y VALIDACIÓN

Creado por: Katherine Dayanna Castillo Recinos

MAKO – Simulador Educativo de Cirugía Robótica

Documento técnico de ingeniería biomédica · Versión 1.0 · 10 de noviembre de 2026

Alcance: reproducción de un entorno educativo basado en CoppeliaSim, Python, Lua, Flask, Orthanc, DICOM y YOLO.

Estado documental: reconstrucción técnica basada en el desarrollo reportado. Las versiones exactas, rutas internas, dependencias y fragmentos Lua deben contrastarse con los archivos originales antes de su publicación.

Control documental

Campo	Valor
Nombre del proyecto	MAKO – Simulador Educativo de Cirugía Robótica
Repositorio	Mako-Simulador-Educativo
Sistema de referencia	Windows 11, CoppeliaSim, Python, Orthanc
Ruta original conocida	C:\MAKO_PACS
Carácter del sistema	Exclusivamente educativo; sin uso clínico
Responsable de validación	[Completar]
Versión de simulador / Python / Orthanc	[Registrar versiones reales]

Índice temático

- 1. Propósito, alcance y restricciones
- 2. Arquitectura y flujo de datos
- 3. Inventario de software, hardware y archivos
- 4. Preparación de Windows y entorno Python
- 5. Modelo 3D y configuración en CoppeliaSim
- 6. Lua, GUI y automatización del robot
- 7. Cámaras y captura de imágenes
- 8. Control móvil mediante Flask
- 9. Procesamiento DICOM y Orthanc PACS
- 10. Expedientes ficticios y PDF
- 11. Inteligencia artificial YOLO
- 12. Ejecución integral y pruebas de aceptación
- 13. Seguridad, reproducibilidad y publicación GitHub
- 14. Diagnóstico de errores
- 15. Anexos, parámetros y evidencias

1. Propósito, alcance y restricciones

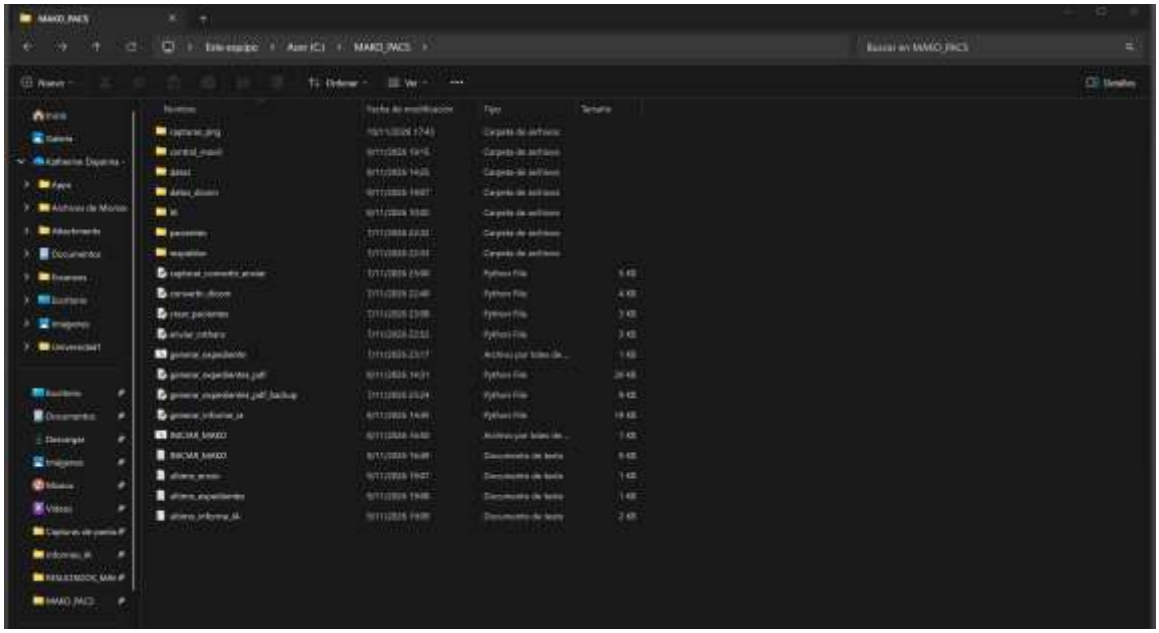
Este manual describe cómo preparar, instalar, configurar, ejecutar y verificar el prototipo educativo de cirugía robótica MAKO. El objetivo técnico es reproducir el flujo desde la simulación de articulaciones hasta el almacenamiento DICOM, análisis experimental y generación de informes. No equivale a documentación de fabricante ni acredita seguridad clínica.

- Simulación del brazo robótico, herramienta y rodilla derecha flexionada.
- Control local mediante interfaz Lua y remoto mediante navegador móvil.
- Tres vistas virtuales y captura PNG.
- Conversión a DICOM y transferencia a Orthanc.
- Generación de expedientes ficticios e informes experimentales con YOLO.

2. Arquitectura y flujo de datos

El diseño utiliza CoppeliaSim como motor de simulación; Lua gestiona la lógica dentro de la escena y los controles de usuario; Python ejecuta procesos auxiliares. Flask ofrece la interfaz de control móvil. Orthanc actúa como repositorio PACS y YOLO analiza imágenes experimentales.

```
Usuario (GUI local / navegador móvil)
       |                          |
       v                          v
CoppeliaSim + Lua  <---->  Python / Flask
       |
       v
Cámaras virtuales -> PNG -> Python -> DICOM -> Orthanc PACS
                                  |
                                  +-> YOLO -> imágenes anotadas + informe PDF
Datos de pacientes ficticios -> JSON -> expediente PDF
```



2.1 Interfaces y dependencias

Componente	Entrada	Salida	Verificación
CoppeliaSim	Escena 3D y comandos	Movimiento y cámaras	Escena ejecuta sin errores Lua
Flask	HTTP desde móvil	Comandos hacia simulador	Página accesible en LAN

Captura	Cámara y escena	PNG	PNG legible
Conversión	PNG + metadatos ficticios	DICOM	Archivo DICOM válido
Orthanc	DICOM enviado	Estudio almacenado	Estudio visible en web
YOLO	Imagen y pesos best.pt	Detecciones, anotaciones	Informe con resultados
ReportLab	JSON y resultados	PDF	PDF abre correctamente

3. Inventario de software, hardware y archivos

Antes de instalar, documentar las versiones exactas, arquitectura de 64 bits, requisitos de GPU, librerías y rutas. La computadora utilizada fue una estación Windows con capacidad de modelado y simulación; la reproducción en otra estación puede requerir ajustes.

Software	Función	Versión / evidencia requerida
Windows 11	Sistema anfitrión	winver
CoppeliaSim	Motor robótico	Help / About
Blender	Edición de mallas	About Blender
Python	Automatización y Flask	python --version
Git + Git LFS	Versionado y archivos grandes	git --version; git lfs version
Orthanc	Servidor DICOM / API REST	Versión de Orthanc
Navegador web	Control móvil y PACS	Versión actual
Modelo YOLO	Detección experimental	Pesos y dependencias reales

3.1 Estructura observada de C:\MAKO_PACS

```
C:\MAKO_PACS\
  capturas_png\
  control_movil\
    servidor.py
  datos\
    RESULTADOS_MAKO\ [según flujo de exportación]
    informes_IA\ [verificar ruta efectiva]
  datos_dicom\
  IA\
    runs\detect\segundo_entrenamiento\weights\best.pt
  pacientes\
  respaldos\
  capturar_convertir_enviar.py
  convertir_dicom.py
  crear_pacientes.py
  enviar_orthanc.py
  generar_expedientes_pdf.py
  generar_informe_ia.py
  generar_expedientes_pdf_backup.py
  generar_expediente.bat
  INICIAR_MAKO.bat
  ultimo_envio.txt / ultimo_expedientes.txt / ultimo_informe_IA.txt
```

Nota: algunas subrutas son reportadas por el flujo de trabajo y deben comprobarse con el árbol real. La escena .ttt/.ttm y los modelos Blender pueden encontrarse fuera de C:\MAKO_PACS: deben incorporarse y documentarse para garantizar reproducibilidad.

4. Preparación de Windows y entorno Python

Se recomienda reproducir el sistema en una copia de trabajo y conservar intacta la instalación original. Usar rutas cortas sin caracteres especiales cuando sea posible y verificar permisos de lectura/escritura.

4.1 Comprobaciones iniciales

```
winver
python --version
py -0p
git --version
git lfs version
```

4.2 Crear entorno virtual

```
cd /d C:\MAKO_PACS
py -m venv .venv
.venv\Scripts\activate
python -m pip install --upgrade pip
```

No se dispone de un requirements.txt verificado. La lista siguiente es orientativa y NO sustituye las importaciones reales de los scripts. Instalar solamente paquetes que los archivos requieran y registrar versiones reproducibles.

```
python -m pip install flask requests pydicom pillow reportlab
# Si el código usa YOLO de Ultralytics:
python -m pip install ultralytics
python -m pip freeze > requirements.txt
```

Si el proyecto utiliza bibliotecas adicionales para DICOM (p. ej. pynetdicom) o para interacción con CoppeliaSim, identificarlas en los imports y agregarlas al requirements.txt. No instalar paquetes a ciegas en el entorno original.

4.3 Variables y configuración

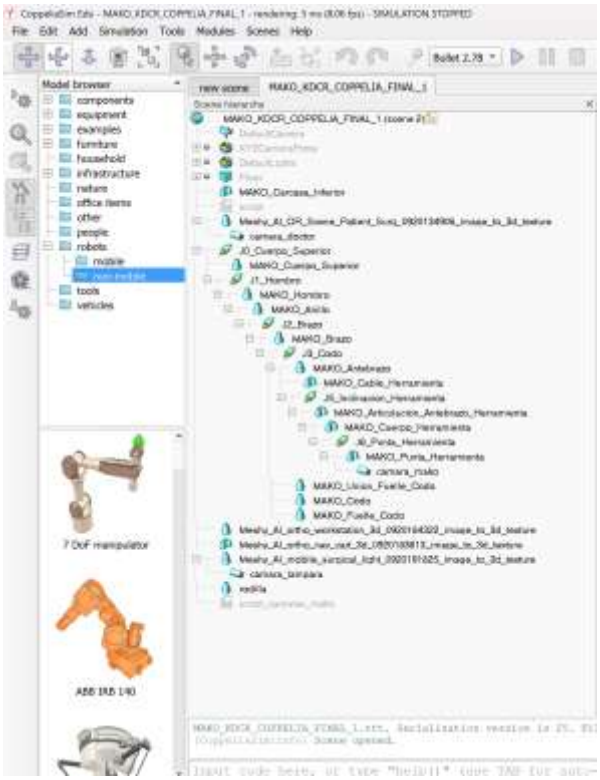
- Verificar rutas absolutas codificadas en los scripts (C:\MAKO_PACS, directorios de captura, datos, pesos YOLO).
- Verificar si el servidor Orthanc usa localhost:8042 y qué autenticación exige.
- Verificar IP/puerto del servidor Flask y la API de CoppeliaSim usada por el control.
- Guardar credenciales en variables de entorno o archivo .env no versionado; nunca publicar contraseñas.
- Comprobar que las carpetas de salida existan y tengan permisos de escritura.

5. Modelo 3D y configuración en CoppeliaSim

La escena incluye el brazo robótico MAKO inspirado en un sistema ortopédico, un paciente con rodilla derecha flexionada, equipo de quirófano y objetos auxiliares. Los elementos importados deben conservar transformaciones, escala, orientación y jerarquía padre-hijo.

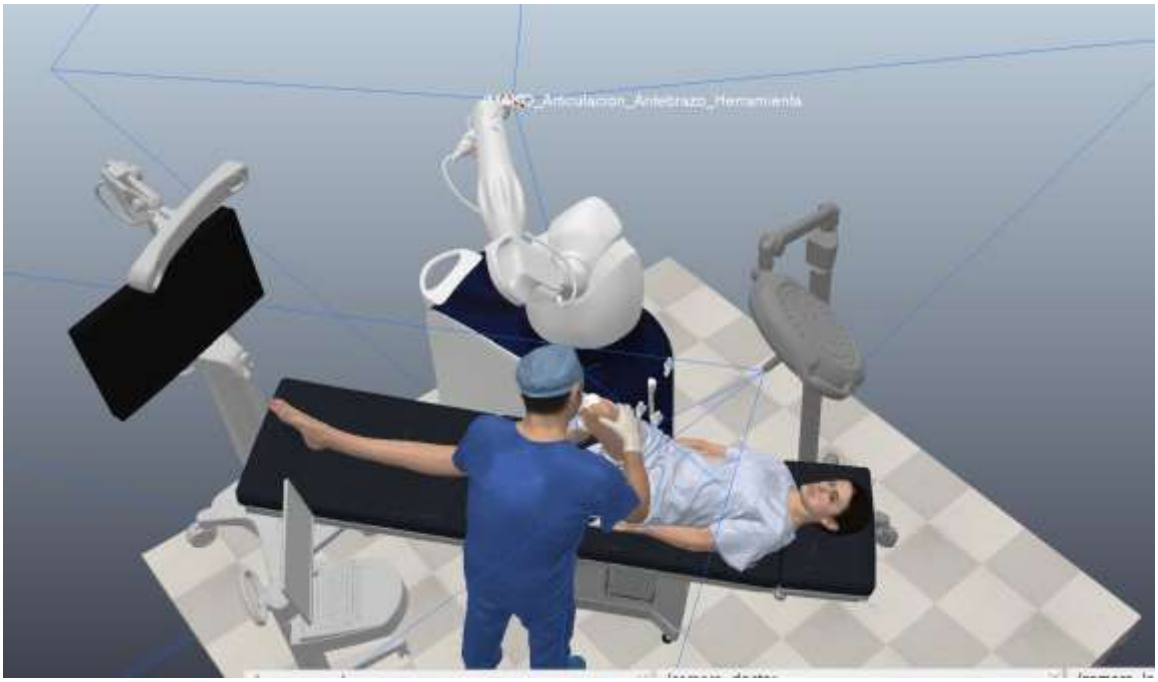
5.1 Componentes reportados

- MAKO_Base_Brazo, MAKO_Cuello_Base, MAKO_Anillo_Base.
- MAKO_ARTICULACION_1, MAKO_Fuelle_Codo, MAKO_Fuelle_Articulacion_1.
- MAKO_Articulacion_Herramienta y MAKO_Fresa.
- Camilla, médico, lámpara, monitores y modelo óseo de rodilla flexionada.



5.2 Reconstrucción de escena

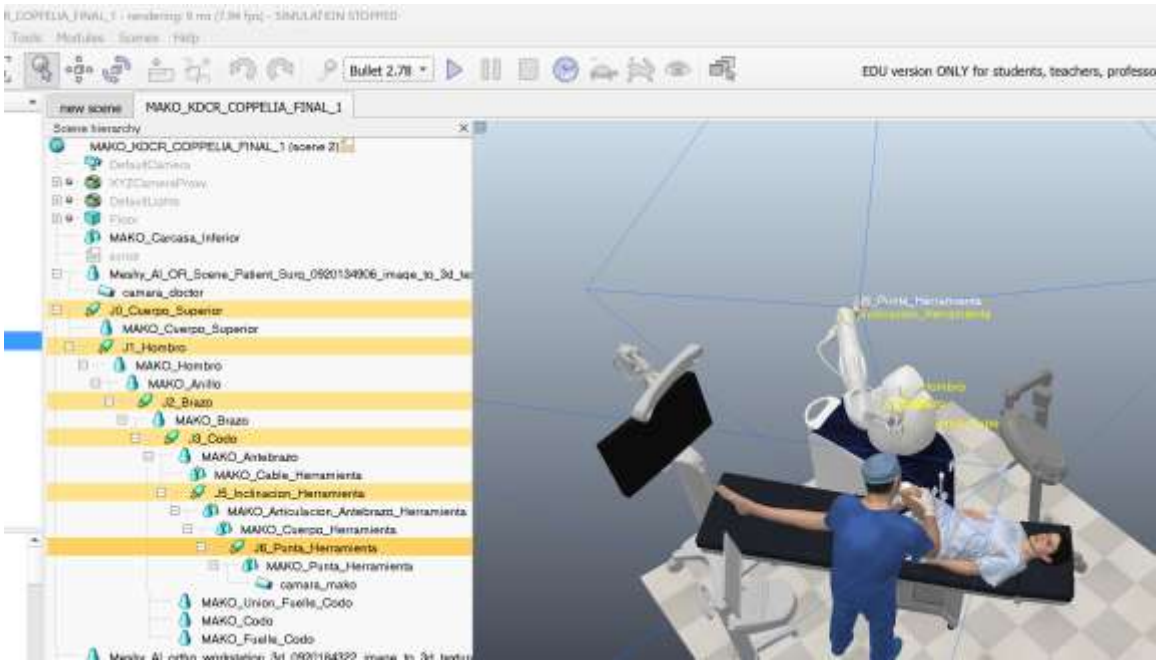
1. Abrir la versión compatible de CoppeliaSim y cargar la escena original (.ttt).
2. Comprobar que todas las mallas estén presentes y no existan referencias externas rotas.
3. Revisar escala de importación, unidades, orientación de ejes y posición relativa de cada objeto.
4. Inspeccionar la jerarquía del árbol de escena y los pivotes de las articulaciones.
5. Verificar que la herramienta esté vinculada a la articulación terminal correspondiente.
6. Guardar una copia versionada antes de cambiar joints o scripts.



5.3 Grados de libertad y articulaciones

Elemento	Tipo reportado	Función / verificación
Elevación de base	Prismática	Desplazamiento de elevación
Cuello	Revoluta	Orientación del conjunto
Movimiento vertical	Prismática	Ajuste de altura
Articulación 1	Revoluta	Orientación del brazo
Inserción	Prismática	Aproximación de herramienta
Articulación terminal / J6	Revoluta en versión final	Oscilación de herramienta

La nomenclatura J0, J1, J2, J3, J5 y J6 se usó en la interfaz de control móvil. No debe suponerse que la correspondencia entre nombres, índices y objetos es idéntica en una escena nueva; inspeccionar handles, límites, modo dinámico/cinemático, signo y unidades de cada joint.



5.4 Pruebas cinemáticas

- Mover cada articulación por separado dentro de sus límites, sin colisiones inesperadas.
- Verificar desplazamiento lineal de prismáticas y rotación de revolutas.
- Confirmar que la herramienta conserve su orientación y no atraviese la anatomía durante la aproximación nominal.
- Registrar posiciones iniciales, límites, velocidades y aceleraciones reales en una tabla de parámetros.

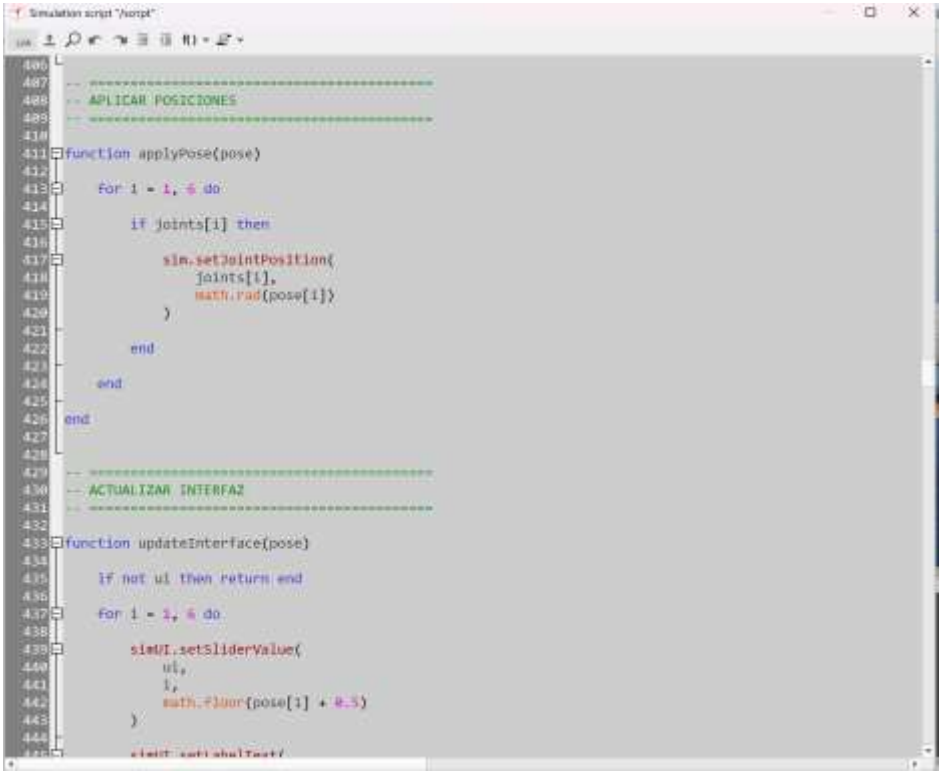
6. Lua, interfaz gráfica y automatización

La interfaz local fue desarrollada mediante scripts Lua y simUI. Incluye controles del robot, ejemplos de movimiento y acciones de captura, envío DICOM, expediente e IA. Las identificaciones de botones y objetos deben tomarse del script real: no se dispone aquí de una copia íntegra verificada.

6.1 Estructura lógica recomendada

```
Inicialización -> resolver handles y construir GUI
Evento de botón -> validar comando -> actualizar objetivo articular
Ciclo de simulación -> ejecutar trayectoria / detener / restablecer
Evento captura -> guardar PNG y actualizar estado
```

```
Evento DICOM / PDF / IA -> invocar proceso auxiliar y registrar resultado
Finalización -> liberar recursos y cerrar interfaz
```



```lua
-- ========================================
-- APLICAR POSICIONES
-- ========================================
function applyPose(pose)
    for i = 1, 6 do
        if joints[i] then
            sim.setJointPosition(
                joints[i],
                math.rad(pose[i])
            )
        end
    end
end

-- ========================================
-- ACTUALIZAR INTERFAZ
-- ========================================
function updateInterface(pose)
    if not ui then return end
    for i = 1, 6 do
        simUI.setSliderValue(
            ui,
            i,
            math.floor(pose[i] + 0.5)
        )
```

6.2 Ejemplos de movimiento

Se desarrollaron tres ejemplos desde la interfaz; uno de los movimientos finales realiza cinco oscilaciones de la herramienta con la secuencia avance → contacto simulado → retroceso → repetición. Documentar los valores exactos de amplitud, velocidad, posición objetivo y temporización a partir del script, evitando inventar valores.

7. Abrir la escena y activar simulación.
8. Seleccionar un ejemplo de movimiento en la GUI.
9. Observar la trayectoria y verificar el punto de contacto con la rodilla.
10. Comprobar que el movimiento de retorno no desplace indebidamente la cámara.
11. Activar Stop y Home; verificar que se detenga y restablezca el estado esperado.





7. Cámaras y adquisición de imágenes

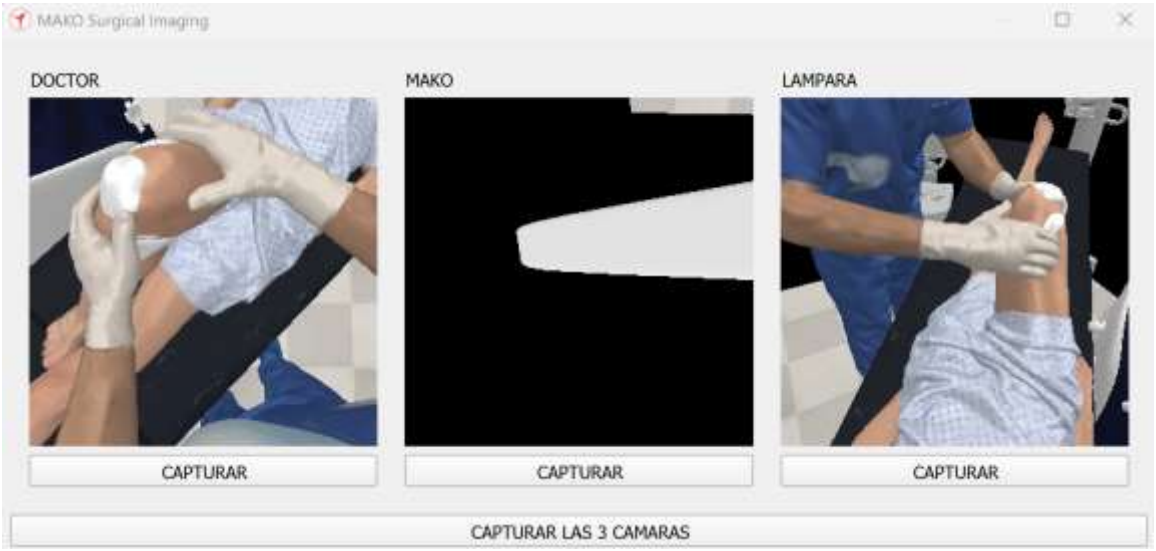
Se configuraron tres cámaras virtuales: camara_doctor, camara_mako y camara_lampara. La vista del doctor se ubicó detrás del operador; la de MAKO enfocó la interacción herramienta-rodilla; la de lámpara mostró el campo quirúrgico. Los monitores virtuales utilizaron texturas estáticas y ventanas de visualización.

7.1 Comprobación de cámaras

12. Inspeccionar nombres de objetos y comprobar que los handles correspondan a las cámaras reales.
13. Ajustar posición y orientación en coordenadas locales o globales según la jerarquía.
14. Comprobar eje óptico y orientación vertical de la imagen.
15. Validar encuadre de rodilla, herramienta y operador durante movimiento.
16. Comprobar la resolución de captura y el formato PNG.

7.2 Captura mediante interfaz

La interfaz simUI incorporó botones “Capturar Doctor”, “Capturar Mako” y “Capturar Ambas”, además de una etiqueta de estado identificada como 203 en la versión reportada. Los nombres de variables en Lua incluyen camDoctor y camMako. Los archivos se guardaron con sellos de tiempo, con nombres tipo doctor_<sesion>.png y mako_<sesion>.png en capturas_png.



7.3 Criterios de calidad

- Imagen legible, sin inversión vertical u horizontal no intencional.
- Herramienta y anatomía dentro del campo de visión.
- Archivo PNG generado con nombre único y marca temporal.
- Ruta de salida consistente con el procesador Python.

8. Control móvil mediante Flask

El control remoto se implementó con un servidor Python Flask en control_movil/servidor.py y una interfaz web accesible desde un teléfono conectado a la misma red local. La interfaz reportada permite actuar sobre J0, J1, J2, J3, J5 y J6, además de Home y Stop.

8.1 Arranque del servidor

```
cd /d C:\MAKO_PACS
.venv\Scripts\activate
python control_movil\servidor.py
```

En el desarrollo se utilizó una dirección de ejemplo http://10.23.146.1:5001. Esta IP depende de la red local y no debe codificarse como universal. Comprobar la IP actual con ipconfig y el puerto real en servidor.py.

```
ipconfig
netstat -ano | findstr :5001
```

8.2 Procedimiento de conexión

17. Iniciar CoppeliaSim y cargar la escena correspondiente.
18. Iniciar el servidor Flask y comprobar que no existan errores de importación.
19. Confirmar que PC y móvil estén en la misma red y que el firewall permita el puerto configurado.
20. Abrir en el móvil http://<IP_LOCAL_PC>:<PUERTO>.
21. Probar un movimiento pequeño por articulación; luego Home y Stop.
22. Verificar latencia, límites de movimiento y ausencia de comandos duplicados.

La forma exacta de comunicación Python–Lua (API remota, señales, socket u otro mecanismo) debe identificarse leyendo servidor.py y el script de CoppeliaSim. No se debe sustituir por un mecanismo distinto sin adaptar ambos extremos.

8.3 Arranque por archivo BAT

Se utilizó INICIAR_MAKO.bat para iniciar el servidor. Revisar que apunte al Python del entorno virtual vigente; los archivos BAT con rutas antiguas dejarán de funcionar al clonar el repositorio en otra PC.

```
@echo off
cd /d "%~dp0"
".venv\Scripts\python.exe" "control_movil\servidor.py"
pause
```

El bloque anterior es una plantilla portable sugerida, no una transcripción del BAT original.

9. Conversión DICOM e integración Orthanc

El flujo validado en el proyecto consiste en generar PNG desde las cámaras, convertirlos a DICOM mediante Python y enviarlos a Orthanc. El sistema de almacenamiento no realiza diagnóstico ni valida por sí solo la exactitud de los metadatos.

9.1 Configuración Orthanc

23. Instalar Orthanc compatible con Windows o iniciar la instancia ya existente.
24. Revisar el archivo de configuración de Orthanc: puerto HTTP, puerto DICOM, autenticación, AE Title y almacenamiento.
25. Iniciar el servicio y abrir http://localhost:8042 desde la computadora anfitriona.
26. Verificar credenciales y disponibilidad de la API REST antes de ejecutar el envío.
27. Comprobar que no existan restricciones de firewall entre los componentes.

Parámetro	Valor de referencia	Acción
URL web / API	http://localhost:8042	Confirmar en Orthanc.json
Puerto DICOM	[Verificar]	Consultar configuración
AE Title	[Verificar]	Consultar configuración
Usuario y contraseña	[No publicar]	Configurar fuera del repositorio
Ruta de almacenamiento	[Verificar]	Asegurar espacio y permisos

9.2 Estructura de datos DICOM

- Identificadores PatientID y PatientName: siempre ficticios.
- StudyInstanceUID, SeriesInstanceUID y SOPInstanceUID: únicos y coherentes por sesión.
- StudyDate/StudyTime, Modality y descripción: documentar valores realmente usados.
- PixelData, Rows, Columns, PhotometricInterpretation y BitsAllocated: consistentes con el PNG convertido.
- Transfer Syntax y SOP Class: verificar compatibilidad con Orthanc y el visor.

La conversión de una captura simulada a DICOM no transforma la imagen en una adquisición diagnóstica real. La modalidad y descripción deben reflejar su naturaleza sintética y educativa.

9.3 Ejecución y comprobación

```
cd /d C:\MAKO_PACS
.venv\Scripts\activate
python capturar_convertir_enviar.py
```

El comando se basa en el nombre del script observado; revisar su interfaz y efectos secundarios antes de ejecutarlo. También existen convertir_dicom.py y enviar_orthanc.py, que pueden utilizarse como etapas separadas si el código lo permite.

28. Generar capturas PNG y verificar los archivos de salida.
29. Ejecutar la conversión y confirmar que aparezcan archivos DICOM en datos_dicom.
30. Verificar que cada DICOM pueda abrirse con una librería compatible y que los metadatos sean ficticios.
31. Enviar al Orthanc configurado y revisar la respuesta del servidor.
32. Abrir la interfaz de Orthanc y localizar el estudio de prueba por su identificador.
33. Comparar el estudio almacenado con la sesión de captura correspondiente.

```
python -c "import pydicom; ds=pydicom.dcmread(r'RUTA_AL_ARCHIVO.dcm'); print(ds.PatientID,
ds.StudyInstanceUID, ds.Rows, ds.Columns)"
```

Control crítico: se reportó que una versión del proceso de IA recuperaba el último estudio de Orthanc. Esto NO garantiza que el estudio corresponda a la sesión actual. Para reproducibilidad debe utilizarse el identificador devuelto al enviar el DICOM o filtrar explícitamente por UID/ID de sesión y verificar coincidencia.



10. Expedientes médicos ficticios y generación PDF

El módulo de expedientes es independiente del proyecto SIGEMI y utiliza exclusivamente pacientes simulados. Se implementó mediante generar_expedientes_pdf.py, con salida de registros JSON, expedientes PDF y un catálogo general. Los resultados se recopilan en datos/RESULTADOS_MAKO según el flujo reportado.

10.1 Flujo de generación

```
Paciente ficticio -> estructura JSON -> plantilla ReportLab -> PDF
                                              |
                                              +-> catálogo / resultados
```

10.2 Verificación

34. Comprobar que la carpeta pacientes contenga solo información sintética.
35. Ejecutar el generador según su interfaz; verificar dependencias ReportLab y rutas.
36. Confirmar que se genere un PDF individual y, si está habilitado, el catálogo general.
37. Verificar identificadores ficticios (p. ej., FICT-MAKO-001), lateralidad de rodilla derecha y fechas coherentes.
38. Comprobar que el PDF abra, tenga páginas legibles y no contenga información personal real.

```
python generar_expedientes_pdf.py
```

11. Inteligencia artificial YOLO e informe experimental

El módulo IA utiliza un modelo YOLO entrenado localmente, con pesos reportados en IA/runs/detect/segundo_entrenamiento/weights/best.pt. El script generar_informe_ia.py analiza imágenes de simulación y vistas derivadas de DICOM, genera imágenes con cuadros delimitadores y un PDF con clases y confianzas.

11.1 Preparación del modelo

39. Verificar que best.pt exista y registrar tamaño, fecha, versión de YOLO y clases entrenadas.
40. Comprobar que la versión de ultralytics/PyTorch sea compatible con los pesos.
41. Inspeccionar las rutas de entrada y salida de generar_informe_ia.py.
42. Confirmar que el procesamiento de Orthanc seleccione el estudio correcto, no simplemente el último disponible.
43. Probar primero con una imagen PNG de referencia del proyecto.

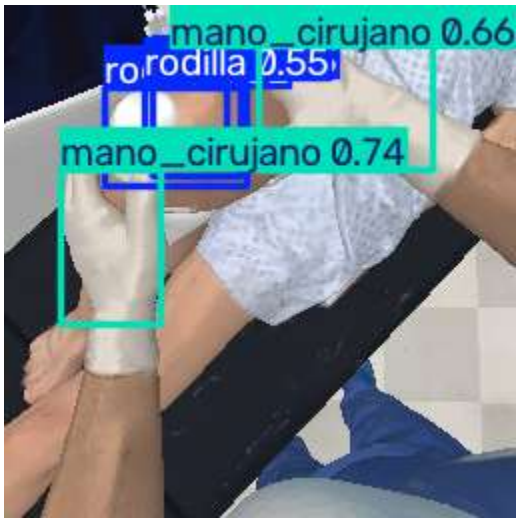
11.2 Ejecución

```
cd /d C:\MAKO_PACS
.venv\Scripts\activate
python generar_informe_ia.py
```

El script puede depender de archivos recién capturados, de Orthanc activo o de rutas configuradas. Revisar las precondiciones en el código antes de su ejecución.

11.3 Salidas y límites

- Imágenes anotadas con bounding boxes y etiquetas de clase.
- Confianzas de detección asociadas a cada objeto identificado.
- Informe PDF multipágina y copias en carpeta de resultados.
- Métricas de precisión y generalización solo si se dispone de conjunto de validación y resultados verificables.
- Las detecciones son experimentales y no constituyen hallazgos diagnósticos.



12. Procedimiento de ejecución integral y pruebas de aceptación

La validación debe ejecutarse con los servicios y módulos en el orden correcto, documentando fecha, versión, resultado esperado, resultado obtenido y evidencia.

12.1 Secuencia operativa

44. Iniciar Windows y comprobar espacio en disco, red local y servicios necesarios.
45. Activar entorno virtual Python; comprobar imports y configuración.
46. Iniciar Orthanc; verificar API HTTP y autenticación.
47. Abrir la escena CoppeliaSim; confirmar modelos, joints y cámaras.
48. Iniciar la simulación y abrir el panel Lua.
49. Iniciar Flask; abrir control móvil desde teléfono en la misma LAN.
50. Probar Home, Stop y movimientos individuales.
51. Ejecutar ejemplo automatizado de cinco oscilaciones.
52. Capturar cámaras Doctor y MAKO; verificar archivos PNG.
53. Ejecutar conversión y envío DICOM; localizar el estudio exacto en Orthanc.
54. Generar expediente ficticio PDF.
55. Ejecutar análisis YOLO e informe experimental.
56. Revisar carpeta RESULTADOS_MAKO y documentar resultados.

12.2 Matriz de aceptación

ID	Prueba	Criterio de aceptación
T01	Carga de escena	Sin mallas faltantes ni errores críticos
T02	Joints	Todos responden dentro de límites
T03	Secuencia	Cinco oscilaciones completas y retorno
T04	Cámaras	Doctor y MAKO con encuadre correcto
T05	PNG	Archivos válidos y con nombre único
T06	Flask	Control accesible desde móvil
T07	DICOM	Metadatos consistentes y píxeles legibles
T08	Orthanc	Estudio correcto almacenado y localizado
T09	Expediente	PDF legible con datos ficticios
T10	YOLO	Detecciones y PDF producidos
T11	Integración	Flujo completo sin intervención manual no documentada

13. Seguridad, reproducibilidad y publicación en GitHub

La publicación del repositorio debe preservar el proyecto y evitar divulgar datos reales, claves, configuraciones locales o archivos innecesarios. El proyecto se clonó como Mako-Simulador-Educativo mediante GitHub Desktop y su directorio local se ubicó en Documentos/github bajo OneDrive.

13.1 Archivos recomendados

- README.md en inglés o español; documentación técnica en docs/documento_tecnico.md.
- Escena de CoppeliaSim, modelos 3D y scripts fuente imprescindibles.
- requirements.txt generado desde entorno validado y un archivo .env.example sin secretos.
- Muestras sintéticas pequeñas de PNG/DICOM y expedientes de ejemplo.
- Capturas y video de demostración optimizados; licencia y guía de usuario.

13.2 Exclusiones y Git LFS

```
# Ejemplo de .gitignore
.venv/
venv/
__pycache__/
*.pyc
.env
.env.*
!.env.example
*.log
*.tmp
Thumbs.db
# Revisar respaldos y datos generados antes de decidir su exclusión
git lfs install
git lfs track "*.ttt" "*.ttm" "*.blend" "*.pt" "*.dcm"
git lfs ls-files
git status
```

Git LFS no elimina los límites de almacenamiento/transferencia de GitHub. Antes de publicar los ~2.60 GB reportados, medir tamaños por archivo, revisar cuotas y considerar una distribución separada para grandes datos de entrenamiento y respaldos. No subir datos de pacientes reales.

13.3 Secuencia GitHub Desktop

57. Clonar el repositorio y conservar intacto C:\MAKO_PACS.
58. Copiar únicamente el contenido seleccionado a la carpeta clonada.
59. Verificar README.md y LICENSE; confirmar extensiones reales de archivos duplicados.
60. Configurar .gitignore y .gitattributes antes del primer commit grande.
61. Revisar Changes, comprobar archivos sensibles y tamaño total.
62. Hacer Commit to main; luego Push origin y verificar en GitHub web.

14. Diagnóstico de errores y recuperación

Los fallos más probables se concentran en rutas absolutas, versiones de dependencias, jerarquías de escena, conectividad local, selección del estudio DICOM y disponibilidad de pesos YOLO.

Síntoma	Causa probable	Diagnóstico / acción
Python no ejecuta	Ruta/venv inválido	where python; activar .venv
Flask no abre en móvil	IP, firewall o puerto	ipconfig; netstat; misma LAN
Joint no se mueve	Handle o modo incorrecto	Revisar jerarquía y Lua
Cámara invertida	Rotación/eje óptico	Revisar orientación y parent
No hay PNG	Ruta o callback	Verificar permisos y log simUI
Orthanc no responde	Servicio o autenticación	Abrir localhost:8042 y revisar logs
DICOM rechazado	Metadatos/transfer syntax	Inspeccionar con pydicom
IA usa estudio equivocado	Selección del último estudio	Usar UID/ID de sesión
No carga best.pt	Ruta o versión incompatible	Verificar archivo y paquete YOLO
Push falla	Archivos grandes o cuotas	Revisar LFS, límites y tamaño

14.1 Estrategia de recuperación

63. Guardar copia de la escena funcional antes de modificar scripts o joints.
64. Registrar mensaje de error exacto y momento de aparición.
65. Reproducir el fallo con el mínimo número de módulos activos.
66. Comparar rutas, versiones y variables contra la instalación validada.
67. Aplicar un cambio a la vez y repetir la prueba correspondiente.

68. Documentar solución y actualizar repositorio con commit descriptivo.

15. Anexos técnicos y registro de evidencias

Completar estas fichas con datos de la instalación original antes de afirmar que el sistema es reproducible en otra estación.

Anexo A. Parámetros por completar

Parámetro	Valor verificado
Versión CoppeliaSim	[Completar]
Nombre y ruta de escena .ttt	[Completar]
Versión Python	[Completar]
requirements.txt y hashes	[Completar]
Flask: host, puerto y rutas API	[Completar]
Mecanismo Python–Lua	[Completar]
Nombres y handles de joints	[Completar]
Posiciones iniciales y límites	[Completar]
Resolución de cámaras	[Completar]
Configuración Orthanc (sin secretos)	[Completar]
Metadatos DICOM usados	[Completar]
Clases YOLO y versión modelo	[Completar]
Entradas/salidas de cada script	[Completar]

Anexo B. Checklist de entrega

- [ ] Escena .ttt y modelos 3D incorporados.
- [ ] Scripts Python y Lua con comentarios técnicos.
- [ ] requirements.txt probado en instalación limpia.
- [ ] Parámetros locales externalizados y documentados.
- [ ] Muestras de datos únicamente ficticios.
- [ ] DICOM vinculados a su sesión de captura.
- [ ] Pruebas T01–T11 ejecutadas y evidenciadas.
- [ ] README, LICENSE, guía de usuario y manual técnico actualizados.
- [ ] Archivos grandes revisados antes de Git LFS / Push.

Declaración final

MAKO es un prototipo académico de simulación e integración tecnológica. Su finalidad es formativa y de investigación; no constituye un sistema clínico validado ni un dispositivo médico certificado. Toda reproducción requiere comprobar las versiones y parámetros reales de la instalación de origen.